UNIVERSIDAD DE CARTAGENA – PROGRAMAS A DISTANCIA

Trabajo Colaborativo Contextualizado. (TCC)

Auditoría de seguridad web y protección criptográfica de credenciales en el portal de una pequeña empresa: detección y mitigación de vulnerabilidades del OWASP Top 10 en un laboratorio aislado

Presentado por (CIPA):

Nelson Ruiz Marimón

Nilson Ruiz Marimón

John Fredy Morales Revueltas

Rafael Alfonso Corredor Gambín

Tutor: Rafael Enrique Benedetti Navarro

Cereté, Córdoba

2026

Tabla de Contenidos

Tabla de Contenidos 2
Descripción de la Problemática 3
Justificación 3
Objetivos 4
Objetivo General 4
Objetivos Específicos 4
Alcance 5
Descripción Detallada de la Solución Planteada 5
Marco teórico 6
Arquitectura del laboratorio aislado 7
Enfoque 1 — Seguridad web (OWASP Top 10) 8
Enfoque 2 — Criptografía aplicada 8
Enfoque complementario — Análisis de riesgos (MAGERIT) 8
Tabla de controles (amenaza → contramedida) 9
Resultados o discusión 9
Evidencias de laboratorio 10
Conclusiones y recomendaciones 12
Bibliografía 13

Descripción de la Problemática

La transformación digital ha llevado a que pequeñas y medianas empresas (PyME) dependan de aplicaciones web para vender, gestionar clientes y almacenar información personal. Sin embargo, en muchas de estas organizaciones el desarrollo y la operación de dichos portales ocurren sin controles de seguridad, por falta de presupuesto, de personal especializado y de conciencia sobre el riesgo. El resultado son aplicaciones expuestas a las vulnerabilidades más comunes y mejor documentadas del mundo, recogidas en el OWASP Top 10.

Para contextualizar el proyecto se toma el caso representativo de **Comercializadora TecnoHogar S.A.S.**, una pequeña empresa local dedicada a la venta de electrodomésticos y artículos para el hogar, que opera un portal web con registro de clientes, inicio de sesión y un panel administrativo para gestionar productos y pedidos. El portal fue desarrollado por un tercero y hoy no cuenta con mantenimiento de seguridad: las contraseñas de los clientes se almacenan sin cifrar, los formularios no validan la entrada y el inicio de sesión no limita los intentos fallidos.

Esta situación expone a la empresa a tres amenazas concretas y frecuentes: la **inyección SQL**, que permitiría a un atacante leer o alterar la base de datos; el **Cross-Site Scripting (XSS)**, que permitiría ejecutar código en el navegador de los clientes y robar sus sesiones; y los **ataques de fuerza bruta** sobre el inicio de sesión, que permitirían adivinar credenciales débiles. A ello se suma un riesgo transversal: al guardar las contraseñas en texto plano, cualquier filtración de la base de datos comprometería de inmediato todas las cuentas.

El problema central que aborda este trabajo es, por tanto: **¿cómo detectar y mitigar las vulnerabilidades web más críticas de un portal de una PyME y proteger adecuadamente las credenciales de sus usuarios, sin poner en riesgo ningún sistema real durante las pruebas?** La respuesta se construye reproduciendo el portal vulnerable en un laboratorio aislado y aplicando, sobre esa réplica, tanto los ataques como sus contramedidas.

Justificación

La seguridad de las aplicaciones web no es un lujo técnico sino una obligación legal y ética. En Colombia, la **Ley 1581 de 2012** y su decreto reglamentario obligan a toda organización

que trate datos personales a adoptar medidas técnicas que garanticen su seguridad y eviten su acceso no autorizado. Una PyME que almacena nombres, correos y contraseñas de sus clientes está sujeta a esta norma, y una filtración podría acarrearle sanciones, además del daño reputacional y la pérdida de confianza.

Las vulnerabilidades elegidas no son teóricas: la inyección y el XSS figuran año tras año en el **OWASP Top 10**, la referencia internacional sobre los riesgos de seguridad en aplicaciones web, y los ataques de fuerza bruta siguen siendo una de las vías más explotadas cuando no hay límite de intentos ni contraseñas robustas. Comprender cómo se ejecutan estos ataques es el primer paso para defenderse de ellos.

El proyecto se justifica, además, porque articula dos enfoques del curso de forma natural y demostrable: la **seguridad web (OWASP Top 10)** y la **criptografía aplicada**. La protección de contraseñas mediante funciones de hash con sal (bcrypt o Argon2) es la contramedida criptográfica que cierra el riesgo de las credenciales en texto plano, de modo que ambos enfoques se refuerzan en un mismo caso. Por último, al realizarse íntegramente en un laboratorio aislado en VirtualBox, el trabajo permite aprender haciendo sin riesgo legal ni técnico para ningún sistema real.

Objetivos

Objetivo General

Auditar las vulnerabilidades web más críticas del portal de una pequeña empresa, reproducido en un laboratorio aislado, demostrando su explotación y aplicando las contramedidas correspondientes, incluida la protección criptográfica de las credenciales, para proponer una configuración segura verificable.

Objetivos Específicos

- Montar un laboratorio aislado con contenedores Docker: un atacante (Kali Linux) y un objetivo deliberadamente vulnerable (DVWA) conectados en una red privada, sin exposición a Internet ni a sistemas reales.

- Detectar y demostrar, mediante herramientas de análisis (Burp Suite, OWASP ZAP, sqlmap e Hydra), las vulnerabilidades de inyección SQL, Cross-Site Scripting (XSS) y fuerza bruta en el inicio de sesión.
- Aplicar las contramedidas de seguridad web correspondientes (consultas parametrizadas, sanitización de salida, cabeceras de seguridad y límite de intentos) y verificar que los ataques dejan de funcionar.
- Implementar la protección criptográfica de las credenciales mediante hash con sal (bcrypt/Argon2), demostrando la diferencia entre el almacenamiento en texto plano y el almacenamiento cifrado.
- Documentar cada vulnerabilidad con su par de evidencia antes–después y elaborar una tabla de controles que relacione cada amenaza con su contramedida.

Alcance

El proyecto abarca la auditoría de tres vulnerabilidades del OWASP Top 10 —inyección SQL, XSS y fuerza bruta— sobre una aplicación web deliberadamente vulnerable (DVWA) montada en un laboratorio aislado, así como la implementación de sus contramedidas y la protección criptográfica de las contraseñas. Se entregan los artefactos de diseño (arquitectura segura, flujo de datos con análisis de amenazas STRIDE y tabla de controles), las evidencias de las pruebas y los scripts utilizados.

Queda fuera del alcance cualquier prueba sobre sistemas, dominios o redes reales; los ataques de denegación de servicio (DoS); el análisis de infraestructura de red (ARP spoofing, MITM), por corresponder a un enfoque distinto; y la auditoría de código fuente completa de una aplicación de producción. Todas las pruebas se realizan exclusivamente contra la máquina víctima del laboratorio.

Descripción Detallada de la Solución Planteada

La solución consiste en reproducir el escenario vulnerable en un entorno controlado y recorrer, para cada amenaza, el ciclo completo de ataque, análisis de causa y mitigación

verificada. La metodología se apoya en dos enfoques complementarios del curso: la seguridad web según OWASP y la criptografía aplicada.

Marco teórico

El **OWASP Top 10** es el documento de referencia internacional, publicado por la Open Worldwide Application Security Project, que recoge los diez riesgos de seguridad más críticos en aplicaciones web. No es una norma de obligatorio cumplimiento, pero funciona como estándar de facto para auditar y priorizar la seguridad de un desarrollo. Este proyecto se concentra en tres vectores que forman parte histórica de ese listado: la inyección, el cross-site scripting y los fallos de autenticación.

La **inyección SQL** ocurre cuando una aplicación construye una consulta a la base de datos concatenando directamente datos suministrados por el usuario, sin separarlos del código de la consulta. Un atacante puede entonces introducir fragmentos de SQL que alteran la lógica original: autenticarse sin credenciales válidas, leer tablas completas o, en casos graves, modificar o borrar datos. La contramedida estándar son las consultas parametrizadas (prepared statements), que envían el código y los datos por canales separados, de modo que la entrada nunca se interpreta como instrucción.

El **Cross-Site Scripting (XSS)** se produce cuando la aplicación devuelve al navegador datos del usuario sin codificarlos, permitiendo que se ejecuten como código en el contexto de la página. En su variante reflejada, el payload viaja en la petición y se refleja en la respuesta inmediata; en la almacenada, queda guardado en el servidor y se sirve a todos los visitantes. Sus consecuencias incluyen el robo de cookies de sesión, la suplantación de usuarios y la modificación del contenido. Se mitiga codificando la salida, aplicando cabeceras como Content-Security-Policy y marcando las cookies como HttpOnly.

Los **ataques de fuerza bruta** consisten en probar sistemáticamente múltiples combinaciones de credenciales hasta acertar. Su éxito depende de dos factores: la ausencia de controles que limiten los intentos y la debilidad de las contraseñas. Las defensas combinan el bloqueo temporal de cuentas, los retardos progresivos, el uso de CAPTCHA y, de forma transversal, el almacenamiento seguro de las contraseñas.

Ese almacenamiento seguro es el núcleo del enfoque de **criptografía aplicada**. Una función de **hash** transforma la contraseña en una cadena de longitud fija de la que no es posible volver a la original. Sin embargo, un hash simple como MD5 es insuficiente: es rápido de calcular —lo que facilita los ataques por diccionario— y, al no usar **sal**, permite emplear tablas precalculadas (rainbow tables) y revela cuándo dos usuarios comparten contraseña. La sal es un valor aleatorio único por usuario que se combina con la contraseña antes de aplicar el hash. Algoritmos como **bcrypt** y **Argon2** incorporan sal automáticamente y son deliberadamente lentos y ajustables, lo que encarece enormemente cualquier intento de descifrado por fuerza bruta. Por eso la recomendación actual (recogida, por ejemplo, en las guías del NIST) es almacenar las contraseñas con estos esquemas y nunca en texto plano ni con hashes sin sal.

Arquitectura del laboratorio aislado

El laboratorio se monta sobre el equipo anfitrión mediante contenedores Docker conectados en una red privada (bridge) llamada labnet, sin salida a Internet ni contacto con la red real. El contenedor **Kali Linux** actúa como estación del analista y reúne las herramientas de prueba: sqlmap, Hydra y nmap por línea de comandos, y Burp Suite u OWASP ZAP desde el equipo anfitrión. El contenedor **víctima** ejecuta DVWA (Damn Vulnerable Web Application) —Apache, PHP y MariaDB—, que simula el portal de la empresa; se publica únicamente en la dirección local 127.0.0.1 del anfitrión (puerto 4280), de modo que no es accesible desde fuera del equipo. Sobre esta aplicación se activan, en la fase de mitigación, los controles de seguridad: cabeceras en el servidor web, validación y consultas parametrizadas en la aplicación, y almacenamiento de contraseñas con hash y sal en la base de datos. El uso de contenedores en lugar de máquinas virtuales no altera el carácter aislado del laboratorio y reduce el consumo de recursos del equipo.

El diseño preliminar de esta arquitectura, el flujo de datos con su análisis de amenazas mediante STRIDE y la tabla de controles se encuentran en los artefactos de diseño del proyecto y se incorporarán como figuras en el documento final.

Enfoque 1 — Seguridad web (OWASP Top 10)

Para cada vulnerabilidad se sigue el mismo procedimiento. En **inyección SQL**, se demuestra primero cómo una entrada como 1' OR '1'='1 evade la autenticación y cómo una consulta UNION extrae usuarios y contraseñas; la causa es la concatenación directa de la entrada en la consulta, y la contramedida son las consultas parametrizadas junto con la validación de entrada. En **XSS**, se inyecta un script que muestra una alerta o lee las cookies de sesión; la causa es la falta de sanitización de la salida, y la contramedida es la codificación HTML, la cabecera Content-Security-Policy y el atributo HttpOnly en las cookies. En **fuerza bruta**, se emplea Hydra o el módulo Intruder de Burp Suite para probar un diccionario de contraseñas contra el login; la causa es la ausencia de límite de intentos, y la contramedida es el bloqueo temporal, el retardo progresivo y el uso de CAPTCHA.

Enfoque 2 — Criptografía aplicada

El control criptográfico central del proyecto es la protección de las contraseñas. Se demuestra el riesgo del almacenamiento en texto plano —donde una sola filtración expone todas las cuentas— y se migra a un esquema de **hash con sal** usando bcrypt (o Argon2). La sal única por usuario impide el uso de tablas precalculadas (rainbow tables) y garantiza que dos usuarios con la misma contraseña produzcan hashes distintos. Se acompaña de un script que muestra el antes y el después, incluyendo la verificación del inicio de sesión sin almacenar la contraseña en claro. Este control complementa la defensa contra la fuerza bruta y protege las credenciales aun si un ataque de inyección lograra volcar la base de datos.

Enfoque complementario — Análisis de riesgos (MAGERIT)

Como refuerzo de los dos enfoques principales, el proyecto incorpora un análisis de riesgos formal siguiendo la metodología **MAGERIT** (alineada con la norma ISO 27005). Se elaboró un inventario de activos (base de datos de usuarios, credenciales, aplicación web, sesiones, servidor y datos personales de los clientes), se identificaron las amenazas y vulnerabilidades por activo, y se valoró cada riesgo como el producto de su probabilidad por su impacto, clasificándolo en nivel alto, medio o bajo. El resultado muestra que los riesgos de mayor nivel —inyección SQL, XSS, fuerza bruta, credenciales sin cifrar y fuga de datos

personales— coinciden con las vulnerabilidades que el proyecto demuestra y mitiga, y que tras aplicar los controles el riesgo residual de cada uno desciende a nivel bajo. El análisis completo y la matriz de riesgos cuantitativa se incluyen como material del proyecto (carpeta analisis-riesgos).

Tabla de controles (amenaza → contramedida)

La siguiente tabla resume la relación entre cada amenaza identificada y el control que la cierra; cada fila se verifica en el laboratorio con evidencia antes–después.

Resultados o discusión

Nota de avance (31 de octubre de 2026): esta sección recoge los resultados preliminares alcanzados a la fecha de entrega del avance. Los resultados completos, con el par de evidencias antes–después de cada ataque, se consolidarán para la entrega final del 20 de noviembre de 2026.

A la fecha se ha completado el diseño del proyecto (arquitectura segura, flujo de datos con análisis STRIDE y tabla de controles), se ha montado el laboratorio aislado con Docker —con los contenedores de Kali y DVWA operativos y comunicados por la red privada— y se han ejecutado las primeras pruebas técnicas de dos enfoques.

En el enfoque de **criptografía**, se ejecutó un script que contrasta el almacenamiento de contraseñas en texto plano frente al hash con sal. El resultado confirma la hipótesis: con las contraseñas en texto plano, las credenciales débiles presentes en un diccionario se recuperan de forma instantánea, mientras que con hash y sal cada intento obliga a recomputar el hash por cada cuenta y las contraseñas robustas no se recuperan con el diccionario de prueba. Se observó, además, que DVWA almacena sus contraseñas como MD5 sin sal, un esquema débil que ilustra precisamente el problema que la mitigación corrige.

En el enfoque de **seguridad web** se demostraron las tres vulnerabilidades elegidas sobre DVWA en nivel de seguridad bajo, y a continuación se verificó su mitigación repitiendo cada ataque en nivel alto o con el código seguro. Los resultados, con su evidencia en la carpeta del avance, son los siguientes:

- **Inyección SQL:** la entrada 1' OR '1'='1 evadió la lógica de la consulta y devolvió los cinco usuarios; una consulta UNION exfiltró los nombres de usuario y sus hashes de contraseña. Tras aplicar consultas parametrizadas (nivel seguro), el mismo payload devolvió cero registros.
- **Cross-Site Scripting (XSS):** el payload <script>alert('XSS')</script> se reflejó y se almacenó sin sanitizar, ejecutándose en el navegador. Con el filtrado y la codificación de salida (nivel alto), la etiqueta <script> fue eliminada y el código dejó de ejecutarse.
- **Fuerza bruta:** un barrido automatizado de diccionario sobre el login halló la contraseña del usuario admin. Con el control de nivel alto —token anti-CSRF por intento y retardo— los intentos automatizados fueron rechazados.

En conjunto, la evidencia antes–después confirma que cada control reduce efectivamente el riesgo de su amenaza. Para la entrega final se consolidarán estas pruebas con capturas adicionales de la interfaz, se ampliará la discusión cuantitativa del riesgo residual y se completará la redacción hasta el mínimo de palabras exigido por la plantilla.

Evidencias de laboratorio

A continuación se incluyen las capturas que respaldan los resultados descritos. El conjunto completo de evidencias (archivos de texto, registros y capturas) se encuentra en la carpeta del proyecto.



Figura 1. Contenedores del laboratorio (Kali atacante y DVWA víctima) en ejecución.

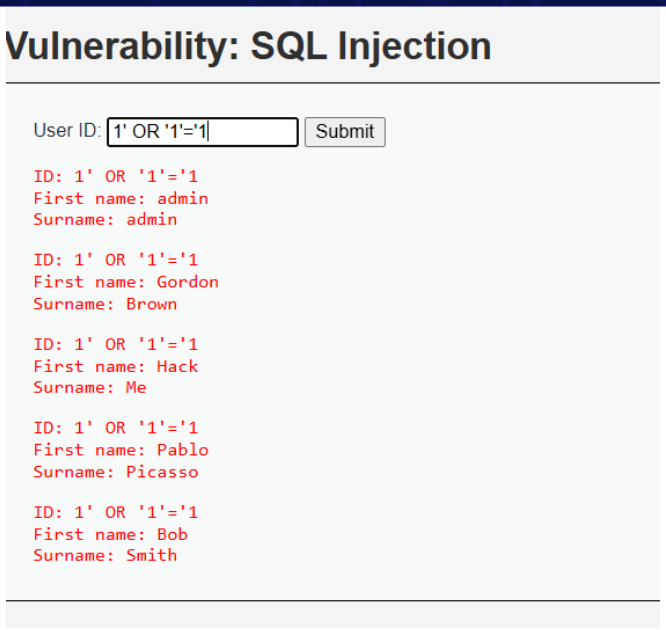


Figura 2. Inyección SQL: la entrada 1' OR '1'='1 devuelve todos los usuarios.

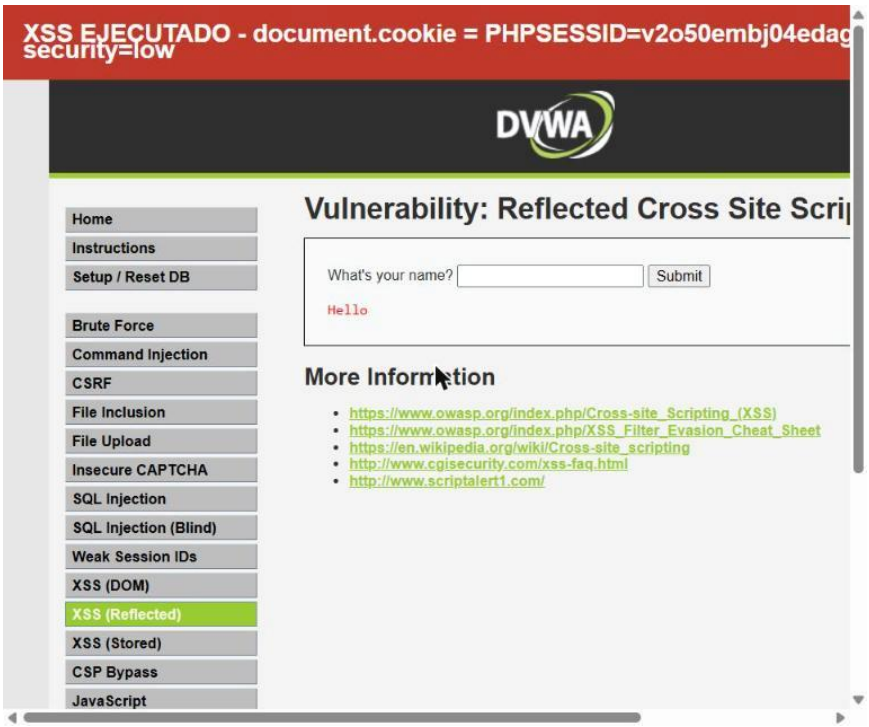


Figura 3. XSS ejecutado: el script inyectado corre y lee la cookie de sesión.

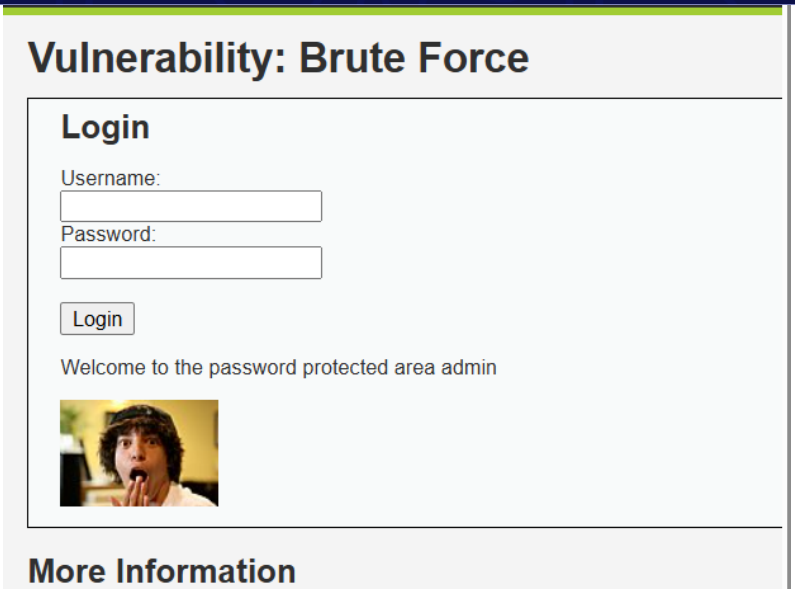


Figura 4. Fuerza bruta: acceso logrado con la credencial hallada (admin / password).

Conclusiones y recomendaciones

El trabajo demuestra que las tres vulnerabilidades más frecuentes en aplicaciones web de pequeñas empresas no solo son fáciles de explotar cuando no existen controles, sino también sencillas de cerrar cuando se aplican las contramedidas adecuadas. En el laboratorio, un atacante sin privilegios logró extraer toda la base de usuarios mediante inyección SQL, ejecutar código en el navegador mediante XSS y adivinar una contraseña débil por fuerza bruta; tras aplicar consultas parametrizadas, codificación de salida y control de intentos, los mismos ataques dejaron de funcionar.

Se confirma además la importancia del enfoque criptográfico: almacenar las contraseñas con hash y sal (bcrypt o Argon2) convierte una eventual filtración de la base de datos en un incidente mucho menos grave, porque las credenciales no quedan expuestas en claro. La combinación de ambos enfoques —seguridad web y criptografía— ofrece una defensa en profundidad, en la que varias capas se refuerzan entre sí.

Como recomendaciones para una organización como la del caso, se proponen: adoptar un ciclo de desarrollo que contemple la seguridad desde el diseño; validar y sanear toda entrada del usuario; utilizar siempre consultas parametrizadas; cifrar las credenciales con algoritmos modernos de hash con sal; aplicar cabeceras de seguridad y políticas de contraseñas robustas; y realizar auditorías periódicas apoyadas en referencias como el OWASP Top 10. Estas

medidas, de bajo costo relativo, elevan de forma notable el nivel de protección y ayudan a cumplir con las obligaciones de la Ley 1581 de 2012.

La principal lección del proyecto es que la seguridad no es un producto sino un proceso: se trata de entender cómo piensa un atacante para poder defenderse, y de respaldar cada decisión con evidencia verificable, tal como se ha hecho en este laboratorio.

Bibliografía

OWASP Foundation. (2021). OWASP Top 10:2021. Recuperado de https://owasp.org/Top10/

Congreso de la República de Colombia. (2012). Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial.

National Institute of Standards and Technology. (2017). NIST Special Publication 800-63B: Digital Identity Guidelines. NIST.

Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference.

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7.ª ed.). Pearson.

OWASP Foundation. (s. f.). DVWA — Damn Vulnerable Web Application. Recuperado de https://github.com/digininja/DVWA

Microsoft. (2009). The STRIDE Threat Model. Microsoft Developer Network.